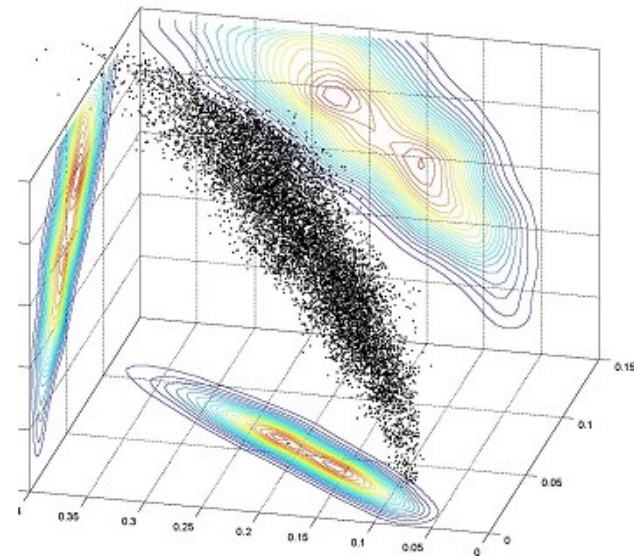
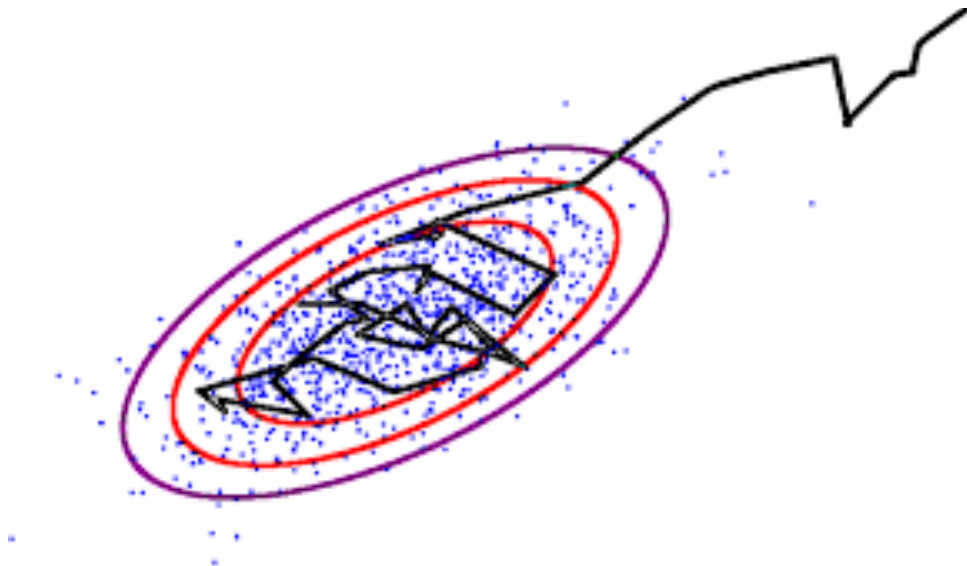
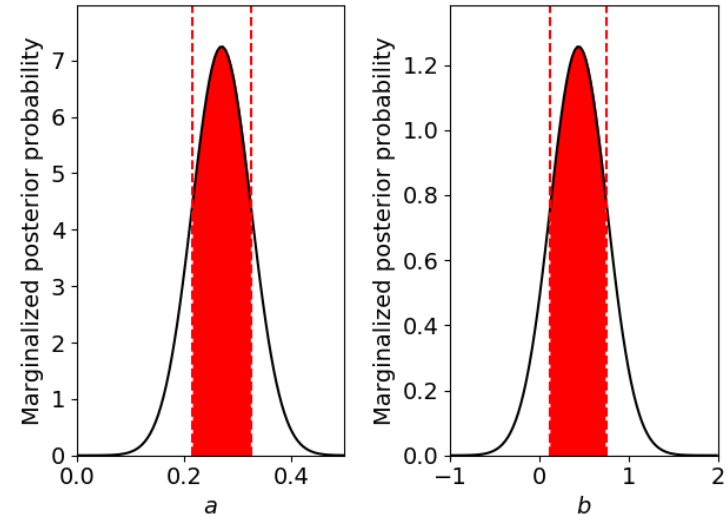
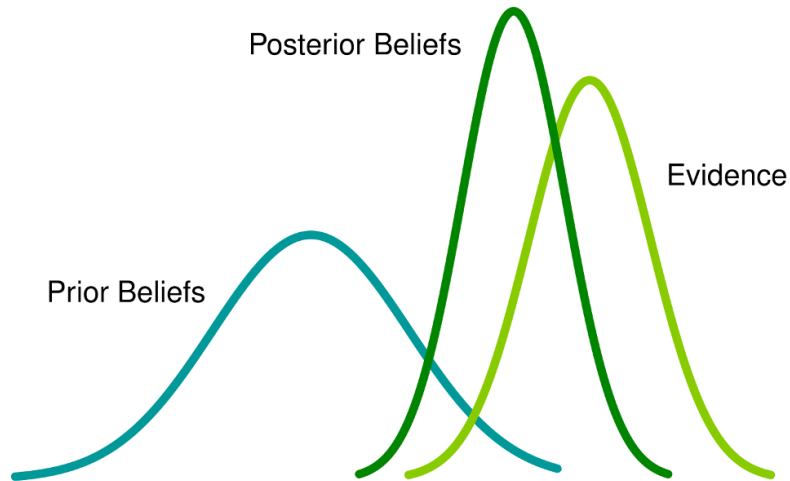
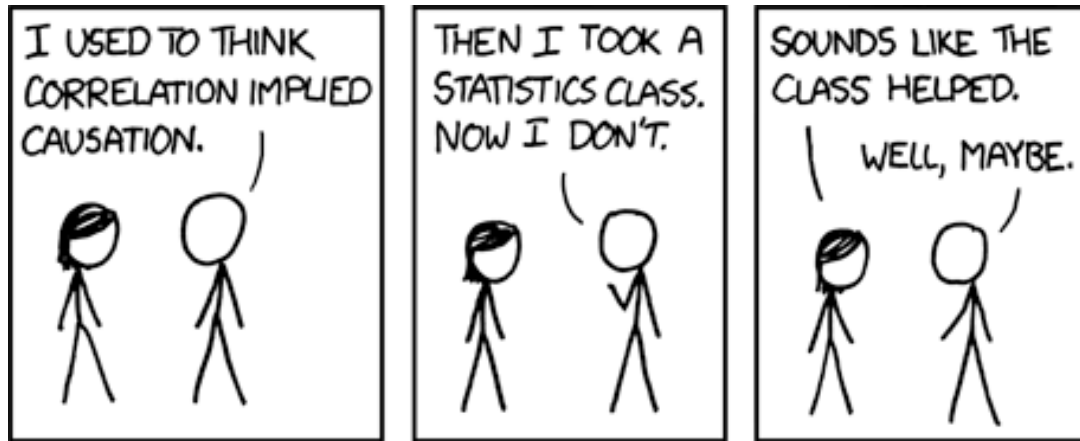


Class 6: Bayesian Methods



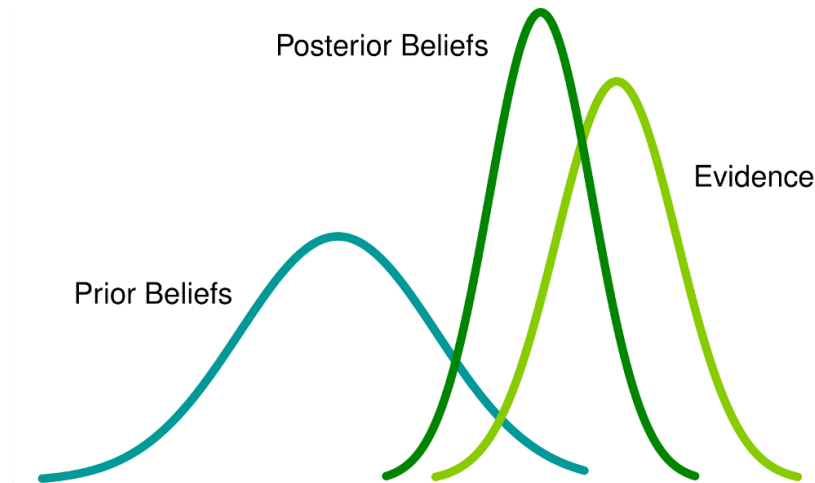
Class 6: Bayesian Methods



- Week 1: Probability and statistics
- Week 2: Correlation testing
- Week 3: Model fitting
- Week 4: Regression
- Week 5: Error estimates
- Week 6: Bayesian methods

Bayesian Methods

- Bayesian statistics allows us to **assign probabilities to a model**



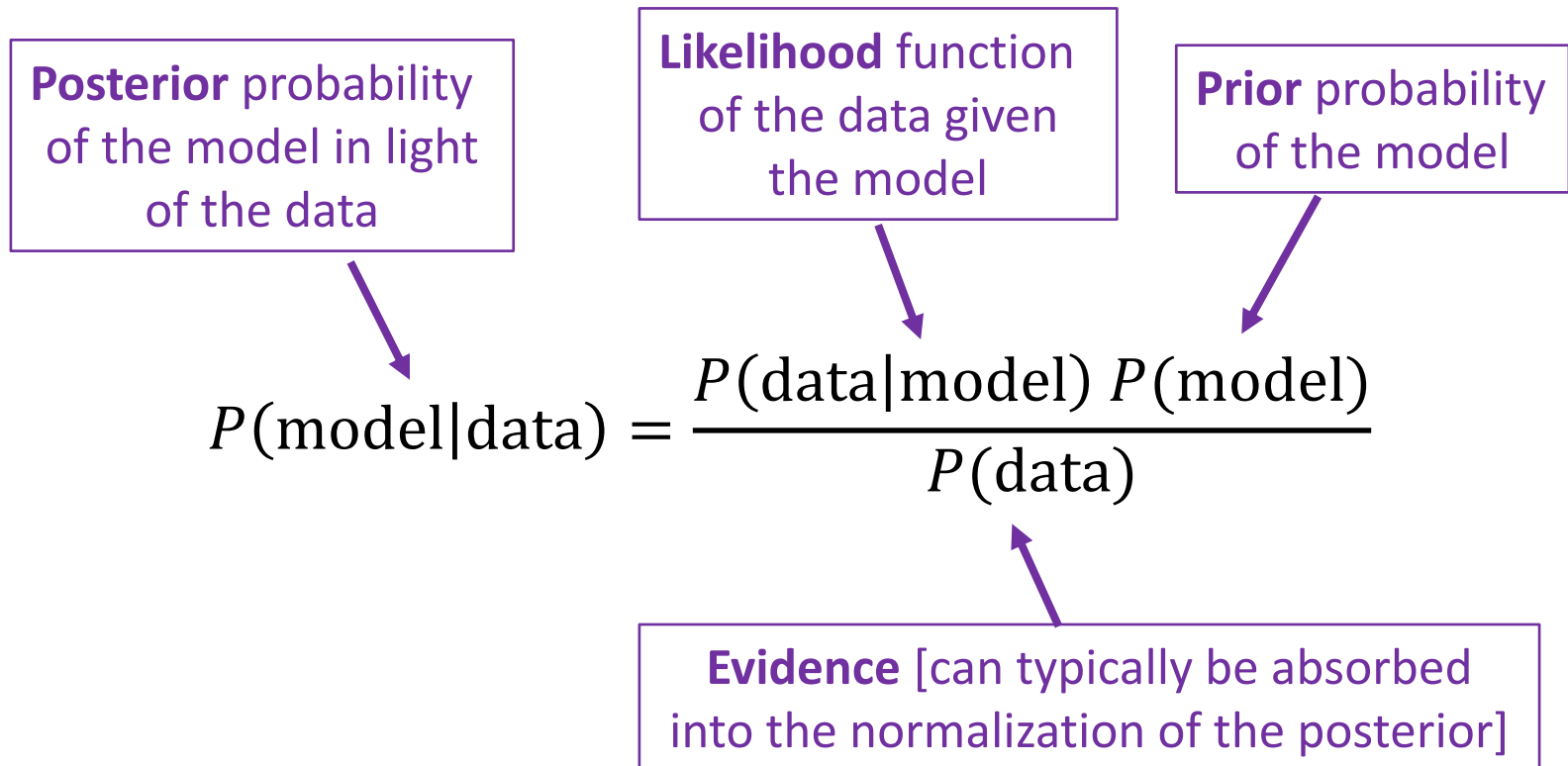
- It makes use of conditional probabilities, $P(A|B)$, meaning “*the probability of A on the condition that B has occurred*”
- *Bayesian statistics* is concerned with **$P(\text{model}|\text{data})$**
- *Frequentist statistics* is concerned with **$P(\text{data}|\text{model})$**

Bayesian Methods

- **Bayes' Theorem** in general:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

- We will typically use it in the form:



Worked Example: Bayesian Methods

- The chance of a certain medical test being positive is 90%, if a patient has disease D . 1% of the population have the disease, and the test records a false positive 5% of the time. If you receive a positive test, what is your probability of having D ?
- We are told: $P(+|D) = 0.9$, $P(D) = 0.01$, $P(+|\text{no } D) = 0.05$
- We want to know: $P(D|+)$
- Bayes' Theorem:
$$P(D|+) = \frac{P(+|D) P(D)}{P(+)} = \frac{P(+|D) P(D)}{P(+|D) P(D) + P(+|\text{no } D) P(\text{no } D)}$$
- Substituting in the data:
$$P(D|+) = \frac{0.9 \times 0.01}{0.9 \times 0.01 + 0.05 \times 0.99} = 0.15$$
- *Interpretation: although the test is correct 90% of the time, the probability of having D after a positive test is only 15%. This is because only a small fraction of the population have the disease.*

Role of the prior

$$P(\text{model}|\text{data}) = \frac{P(\text{data}|\text{model}) P(\text{model})}{P(\text{data})}$$

- Bayesian statistics cannot determine probabilities of a model without assigning a **prior probability**
- The importance of the prior probability is both the strong and weak point of Bayesian statistics
- Strong point: *“the prior probability is a logical necessity which should always be stated”*
- Weak point: *“setting the prior is subjective”*

Role of the prior

- Let's take the example of fitting a parameter a to some data. Bayes' Theorem now reads:

$$P(a|\text{data}) \propto P(\text{data}|a) P(a)$$

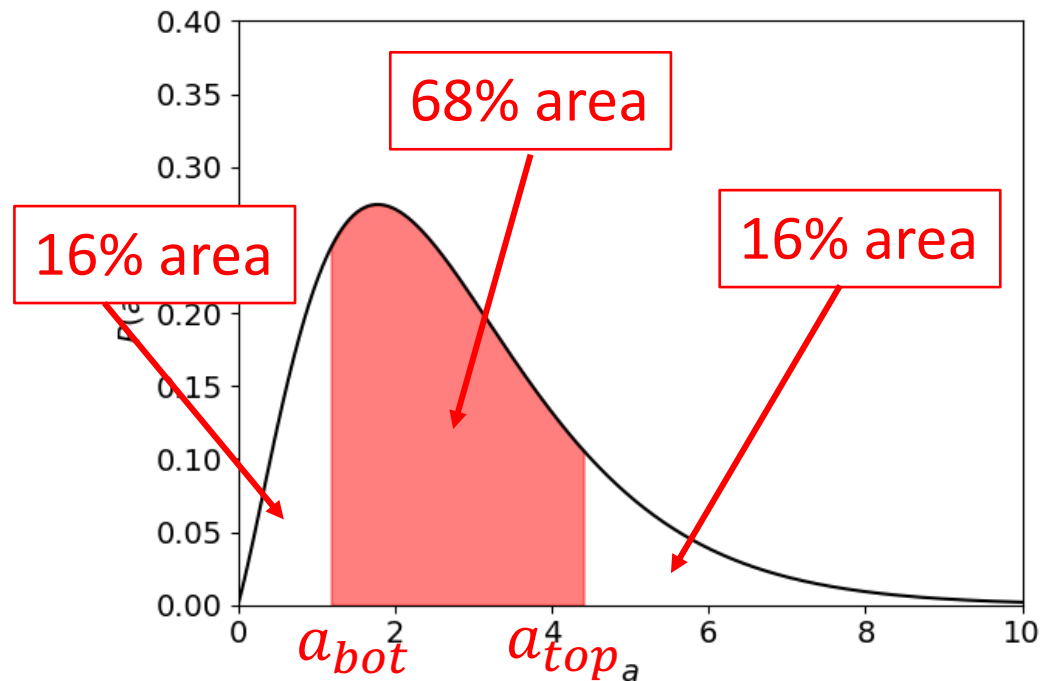
- We do not need the denominator, since we will normalize the posterior $P(a|\text{data})$ such that $\int P(a|\text{data}) da = 1$
- In the absence of other information, a **uniform (or constant) prior** is often assumed for $P(a)$. This is effectively equivalent to the **fitting range** of a parameter
- Assuming Gaussian variables, the **likelihood** $P(\text{data}|a)$ is:

$$P(\text{data}|a) \propto e^{-x^2/2}$$

$$\text{Hence: } P(a|\text{data}) \propto e^{-x^2/2}$$

Posteriors and confidence limits

Result of the Bayesian calculation is the posterior probability distribution:



- For a general probability distribution, we can determine the confidence intervals by integration:

$$\int_{-\infty}^{a_{bot}} P(a) da = 0.16 \quad \int_{a_{bot}}^{a_{top}} P(a) da = 0.68 \quad \int_{a_{top}}^{\infty} P(a) da = 0.16$$

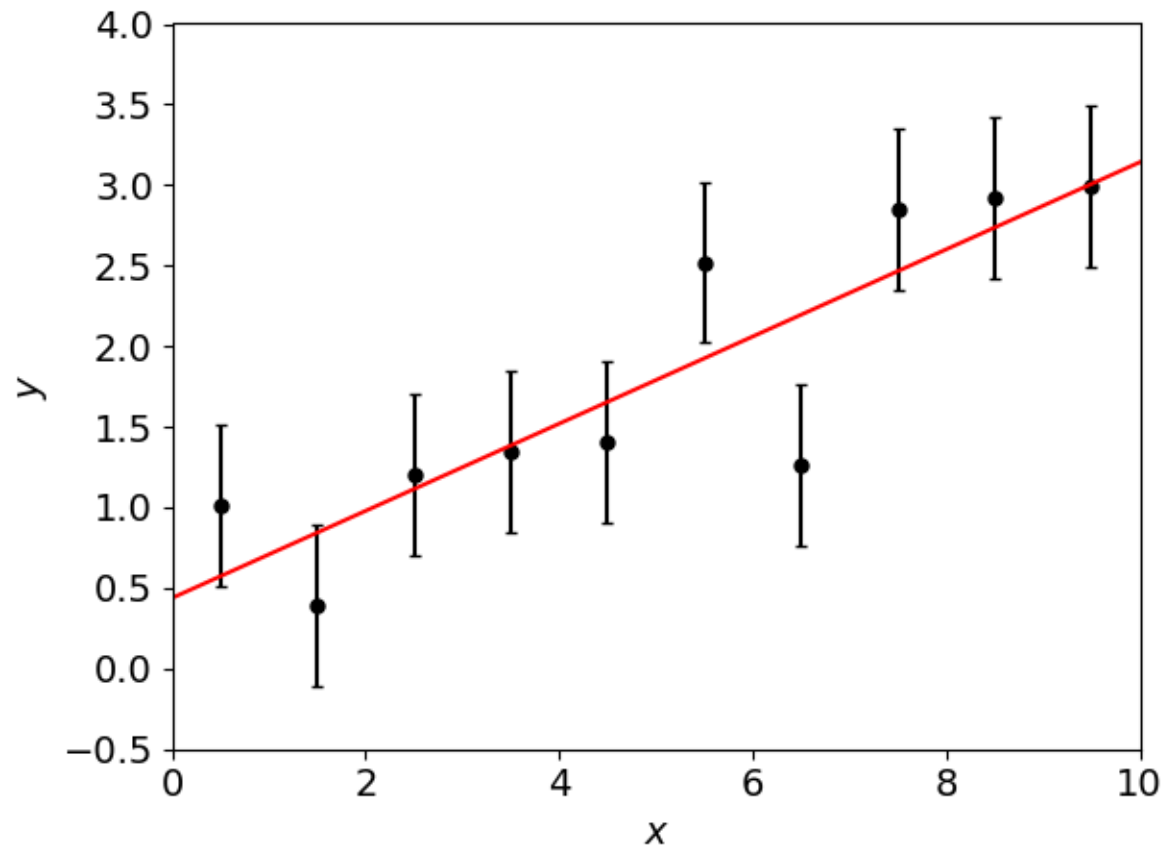
Marginalization

- Suppose we have determined the 2D posterior probability distribution of a 2-parameter fit, $P_{2D}(a, b)$
- *What is the probability distribution for parameter a , considering all possible values of parameter b ?* This is known as **marginalization** of parameter b
- Marginalization can be performed by **summing (integrating) over one axis of the probability distribution**:

$$P_{1D}(a) = \sum_b P_{2D}(a, b)$$

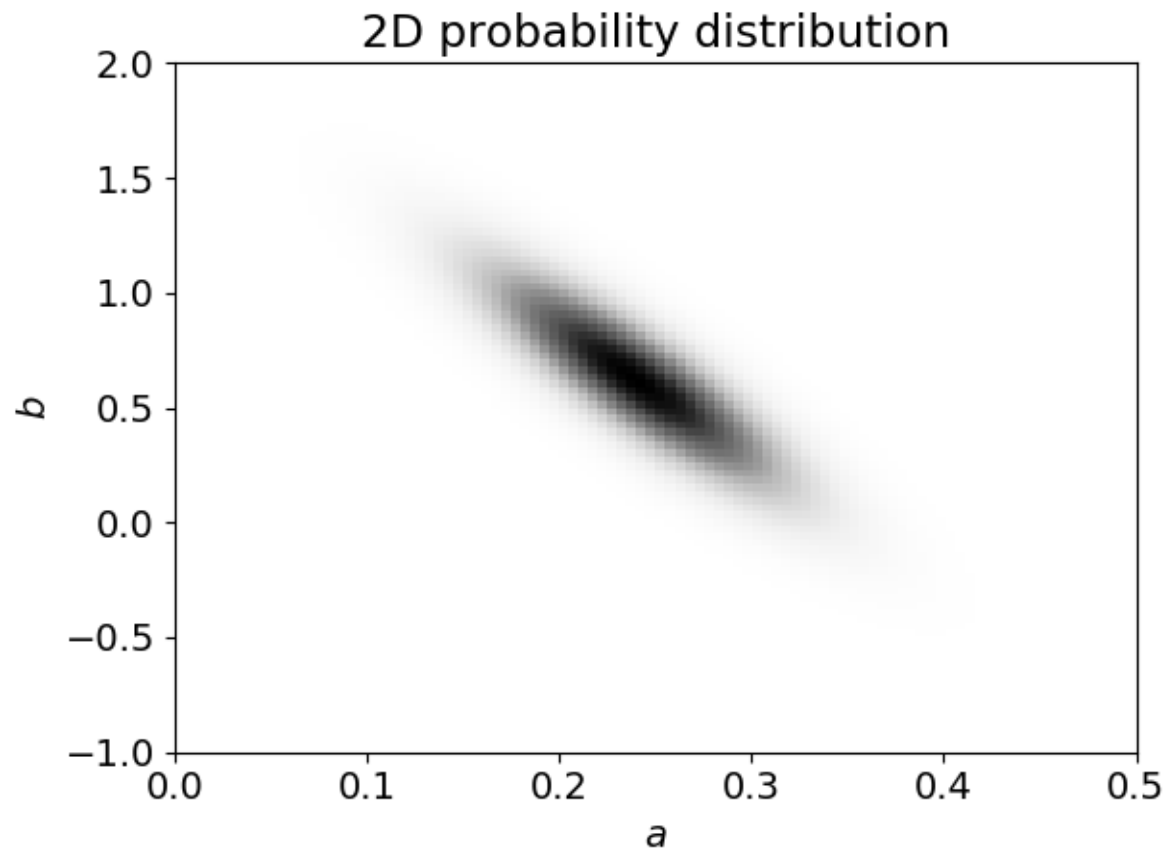
Worked example: parameter fitting

- Let's apply these methods to fitting a straight line $y = ax + b$ to some data...



Worked example: parameter fitting

- Determine the values of χ^2 over a grid of (a, b) and convert to 2D probability $P(a, b) \propto e^{-\chi^2/2}$



Worked example: parameter fitting

Set up parameter grid

Calculate χ^2 for each grid point

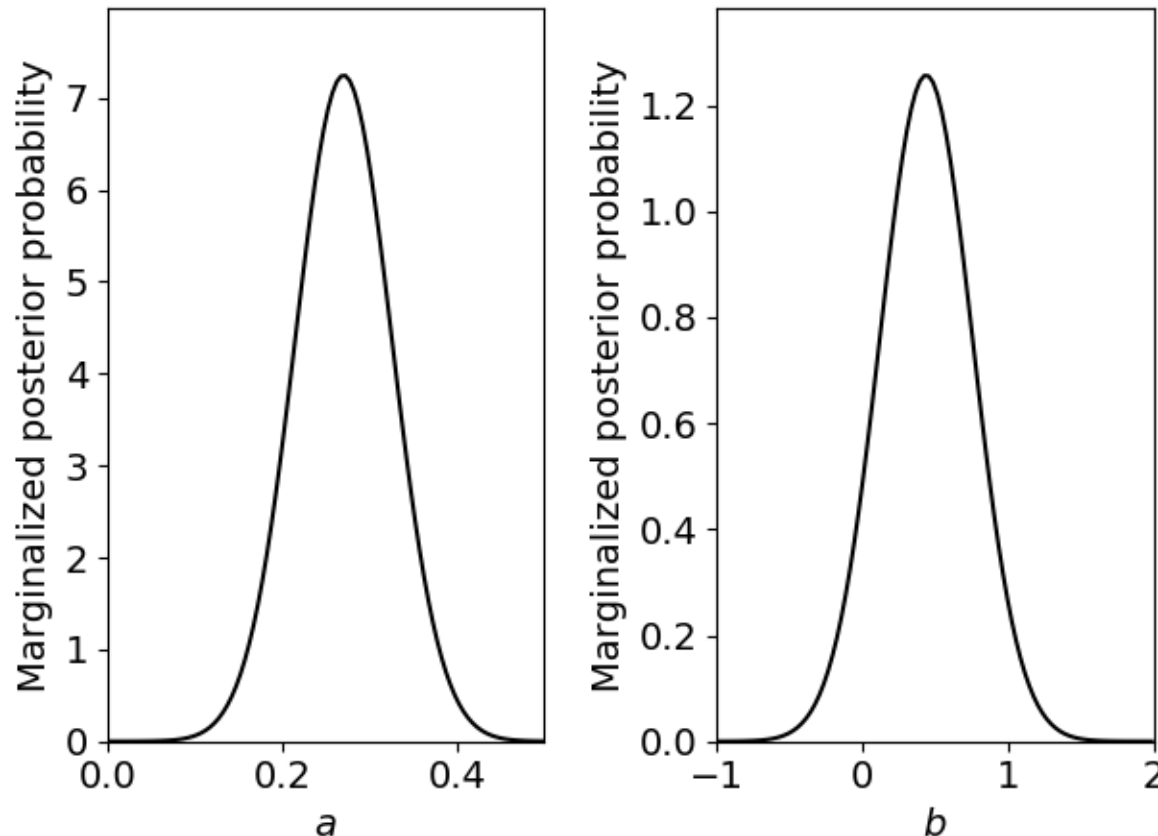
```
# Determine the chi-squared values over a grid of model parameters a,b
# Set the range of parameters a,b and number of grid points
amin,amax,na = 0.,0.5,100
bmin,bmax,nb = -1.,2.,100
da,db = (amax-amin)/na,(bmax-bmin)/nb
alst = np.linspace(amin+0.5*da,amax-0.5*da,na)
blst = np.linspace(bmin+0.5*db,bmax-0.5*db,nb)
# Evaluate the chi-squared value at each grid point (i.e. for each set of model parameters)
chisq2 = np.empty((na,nb))
for i in range(na):
    for j in range(nb):
        chisq2[i,j] = chisqstraightline2([alst[i],blst[j]],x,y,yerr)
# Find the minimum value of chi-squared across the grid
minchisq = np.amin(chisq2)
# Convert the chi-squared values into a probability, using  $P \propto \exp(-\chi^2/2)$ 
# We subtract the minimum chi-squared value in the exponent, to avoid numerical issues of  $\exp(-\text{large number})$ 
prob2 = np.exp(-0.5*(chisq2-minchisq))
# Normalise the probability grid such that  $\sum P(a,b) = 1$ 
prob2 /= np.sum(prob2)
```

Convert χ^2 into probability

Normalise probability

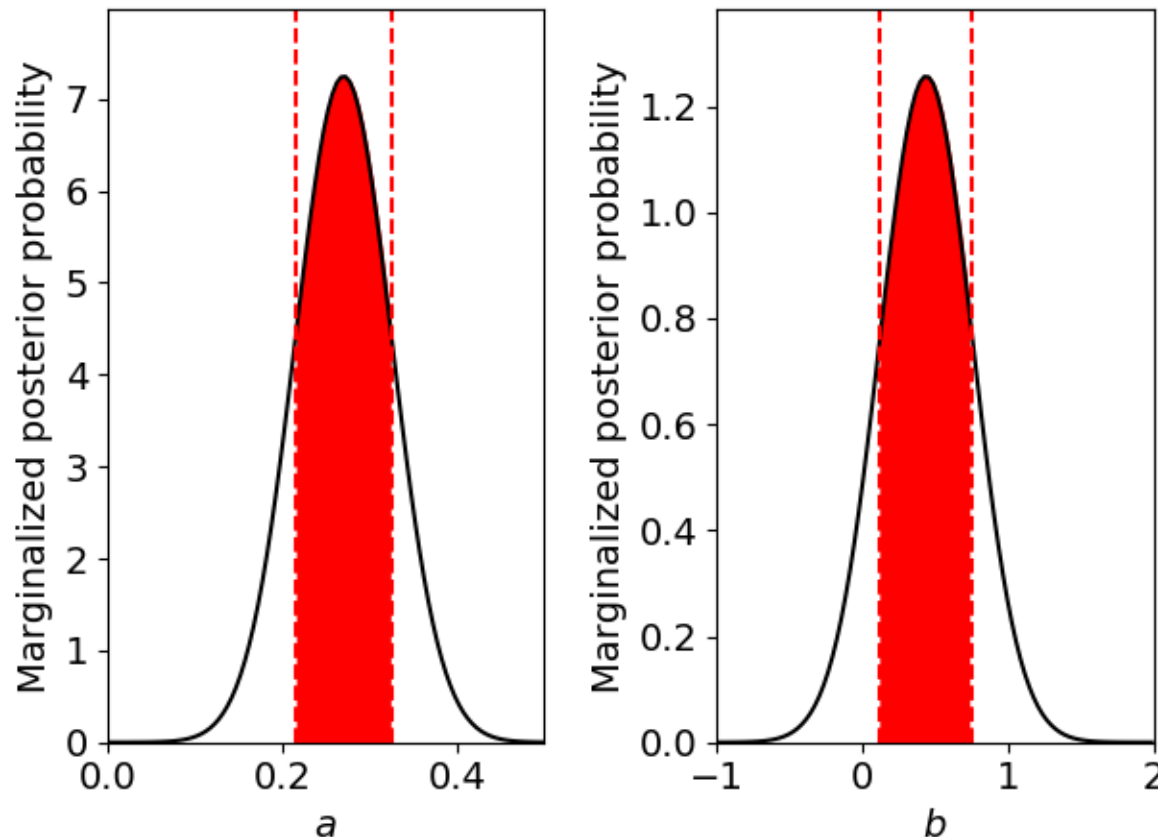
Worked example: parameter fitting

```
# Marginalise the 2D probability distribution to obtain the 1D posterior probabilities of the 2 parameters
# Marginalise parameter b, by summing over the probabilities corresponding to all values of b
proba = np.sum(prob2,axis=1)
# Marginalise parameter a, by summing over the probabilities corresponding to all values of a
probb = np.sum(prob2,axis=0)
# Normalise the probabilities such that  $\int P(a) da = 1$  (currently,  $\sum P(a) = 1$ )
proba /= da
probb /= db
```



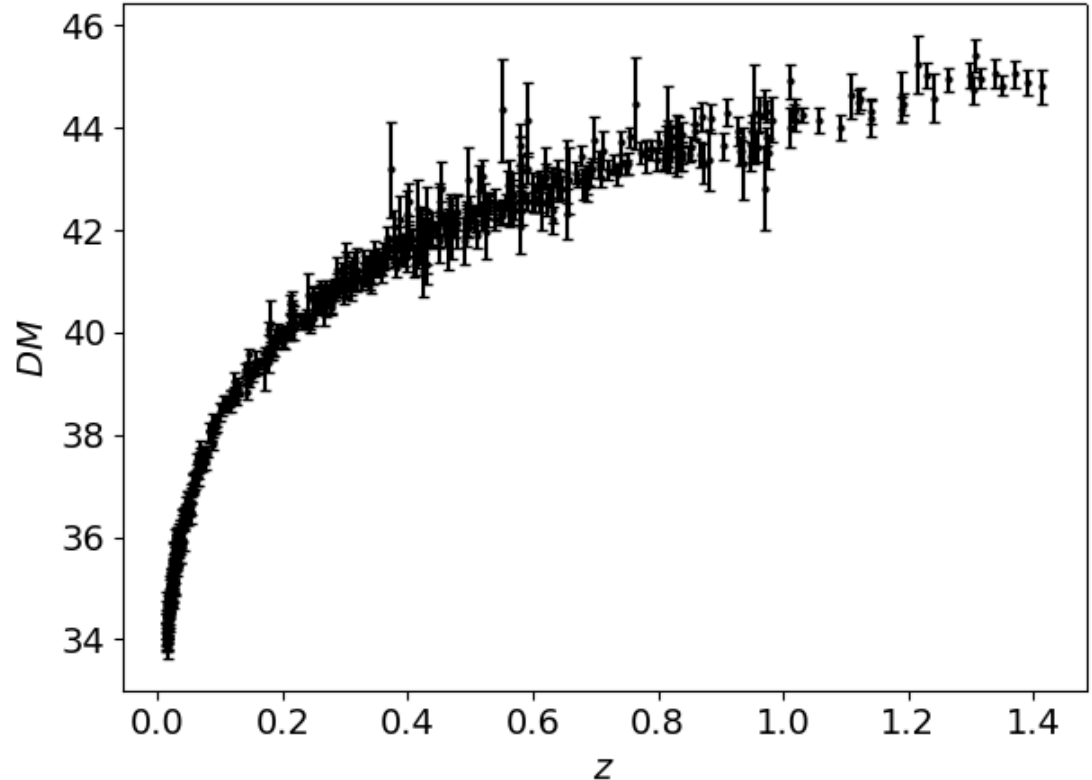
Worked example: parameter fitting

- *By integrating under these distributions, we identify the 68% confidence regions ...*

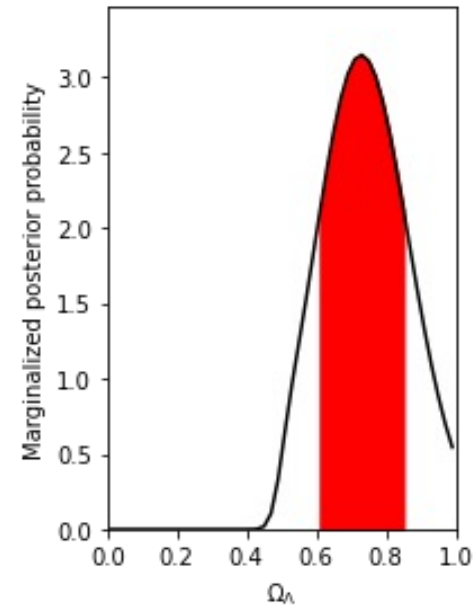
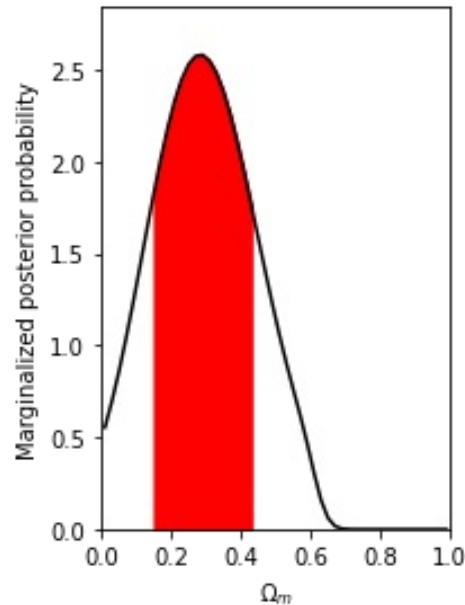
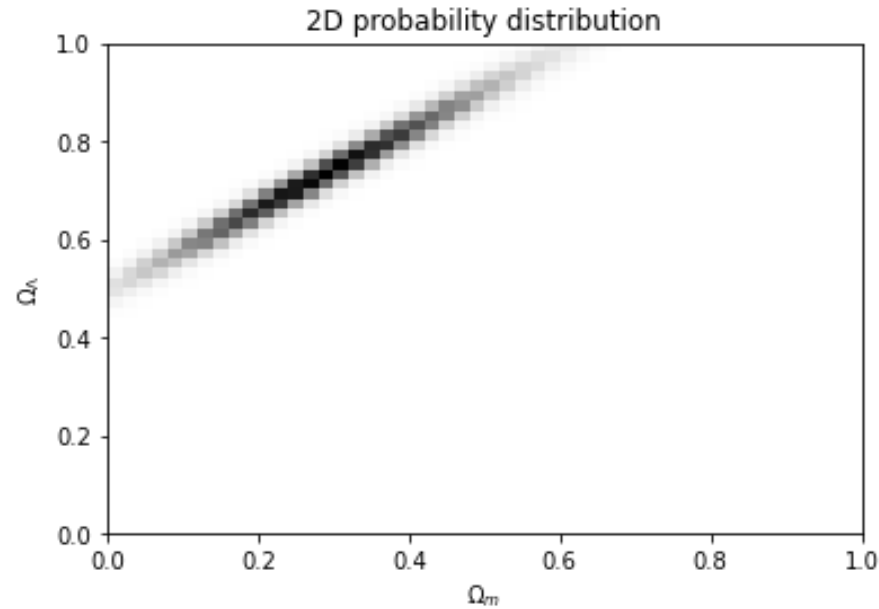


Activity: Likelihood fit

- Let's return to the **supernova distance-redshift dataset**:
- Convert the χ^2 values into a **joint 2D probability distribution** in $(\Omega_m, \Omega_\Lambda)$
- Marginalize this probability distribution to obtain the **1D posterior probability distributions** for Ω_m and Ω_Λ
- Determine the **68% confidence regions** for Ω_m and Ω_Λ



Activity: Likelihood fit



Using prior:

$$0 < \Omega_m < 1$$

$$0 < \Omega_\Lambda < 1$$

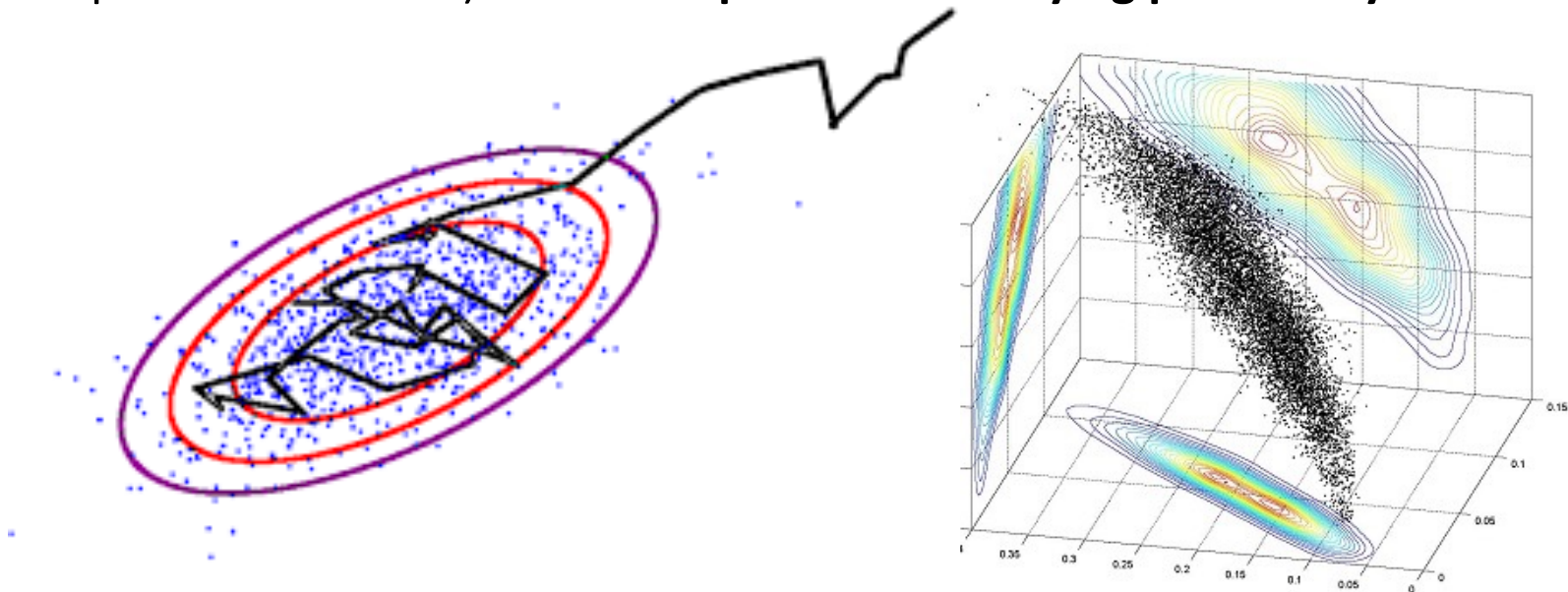
Confidence regions:

$$0.15 < \Omega_m < 0.45$$

$$0.61 < \Omega_\Lambda < 0.86$$

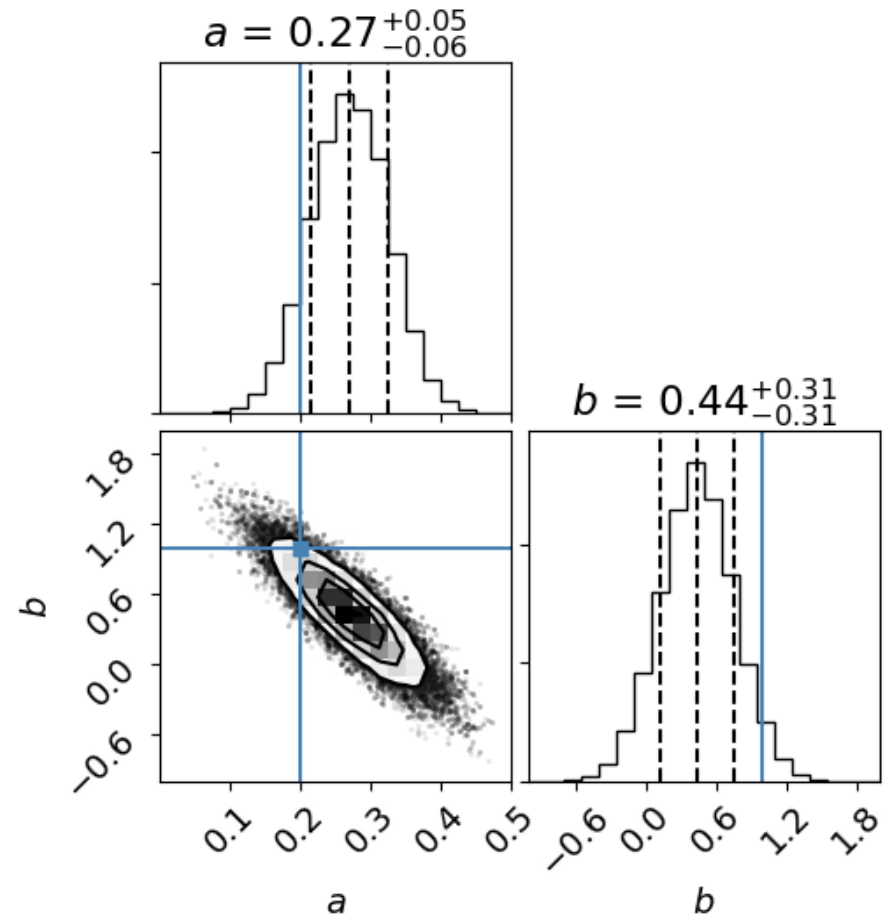
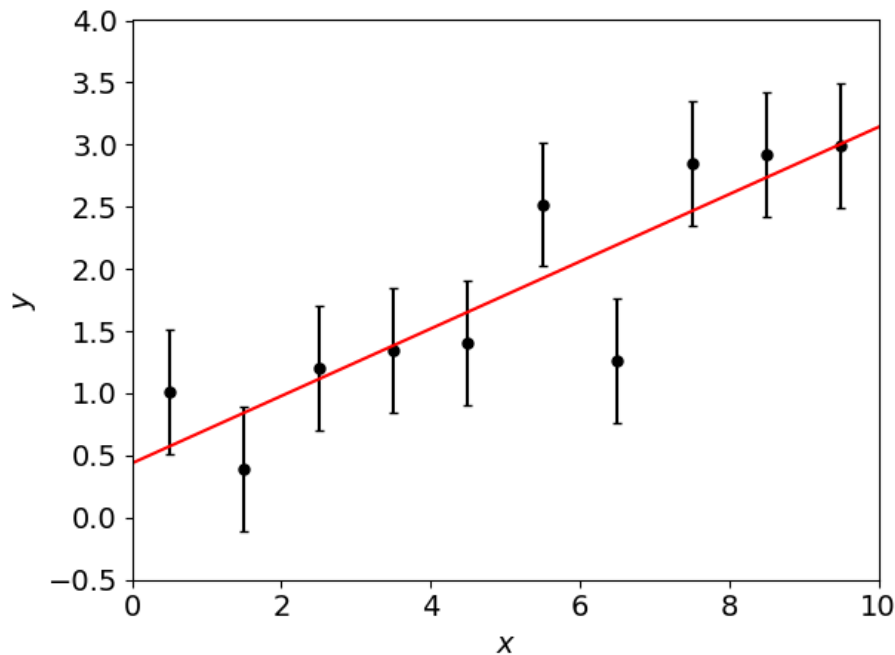
Monte Carlo Markov Chains

- *The grid method becomes inefficient as the number of parameters increases.* A powerful alternative is to generate a **Monte Carlo Markov Chain (MCMC)** in the parameter space
- There are various algorithms to do this such as python *emcee* (we won't go into details here), but the end result is a “chain” (distribution of parameter values) which **samples the underlying probability distribution**



Worked example: MCMC

- Using python's *emcee* algorithm to sample the probability distribution of the straight-line fit:



Worked example: MCMC

```
# Uniform prior
def lnpriorline(p, amin, amax, bmin, bmax):
    a, b = p
    if ((amin < a < amax) and (bmin < b < bmax)):
        return 0.0
    return -np.inf

# Likelihood
def lnlikeline(p, x, y, yerr):
    chisq = chisqstraightline2(p, x, y, yerr)
    return -0.5*chisq

# Posterior
def lnproblne(p, x, y, yerr, amin, amax, bmin, bmax):
    lp = lnpriorline(p, amin, amax, bmin, bmax)
    if not np.isfinite(lp):
        return -np.inf
    return lp + lnlikeline(p, x, y, yerr)

ndim = 2 # Number of parameters
nwalkers = 100 # Number of walkers
a0, b0 = 0.2, 1. # Fiducial values
# Priors
amin, amax = 0., 0.5
bmin, bmax = -1., 2.
# Initialize the chain
pmin = np.array([amin, bmin])
pmax = np.array([amax, bmax])
p = [pmin + (pmax-pmin)*np.random.rand(ndim) for i in range(nwalkers)]
# Set up sampler
sampler = emcee.EnsembleSampler(nwalkers, ndim, lnproblne, args=(x, y, yerr, amin, amax, bmin, bmax))
# Burn-in phase (use 300 steps)
p, prob, state = sampler.run_mcmc(p, 300)
sampler.reset()
# Perform MCMC (use 700 steps)
p, prob, state = sampler.run_mcmc(p, 700)
samples = sampler.flatchain
```

Prior

Likelihood

Posterior =
Likelihood * Prior

Starting point of walkers

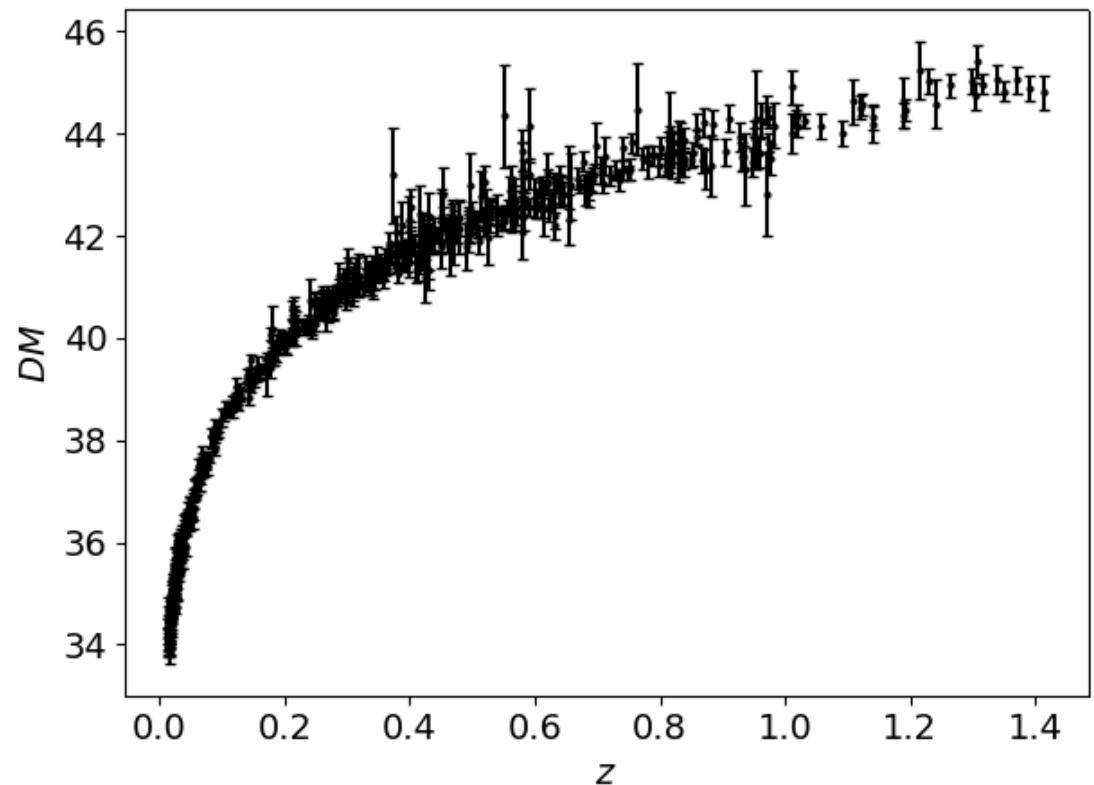
Initialise sampler

Burn-in phase

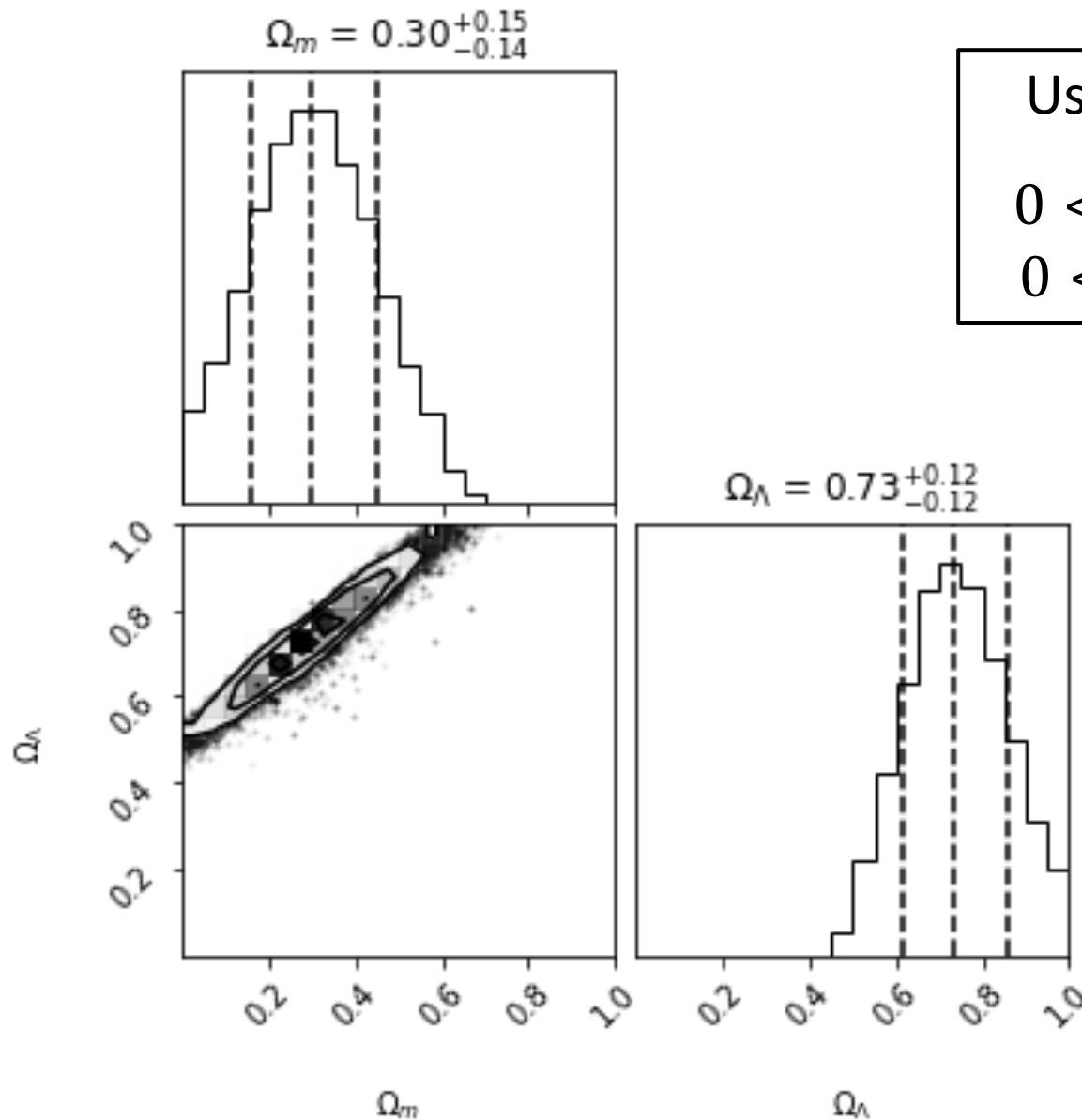
Main run

Activity: MCMC

- Let's return to the same **supernova distance-redshift dataset** again:
- Run an **MCMC analysis** for parameters $(\Omega_m, \Omega_\Lambda)$
- Determine the **68% confidence regions** for Ω_m and Ω_Λ

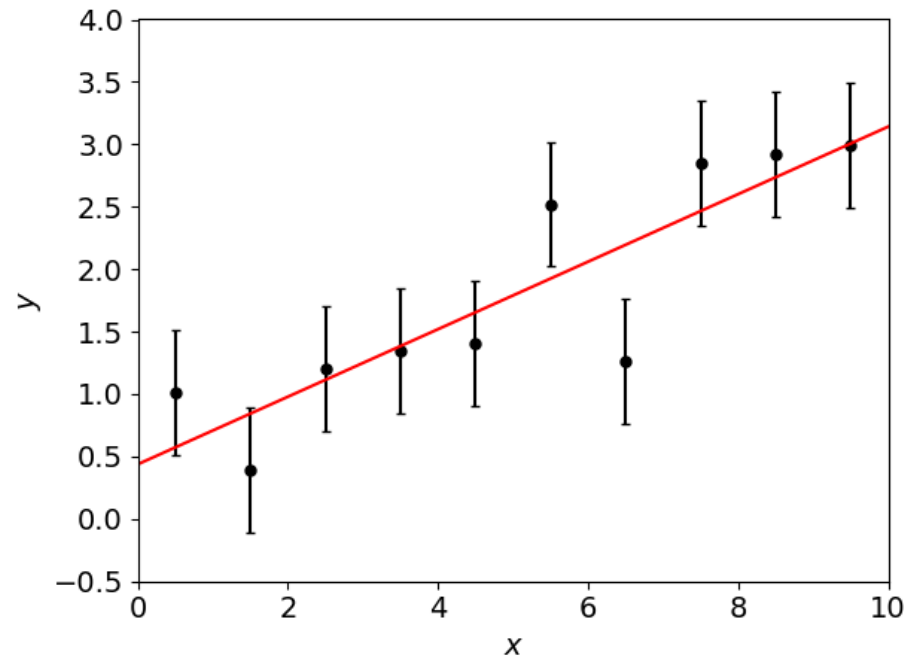
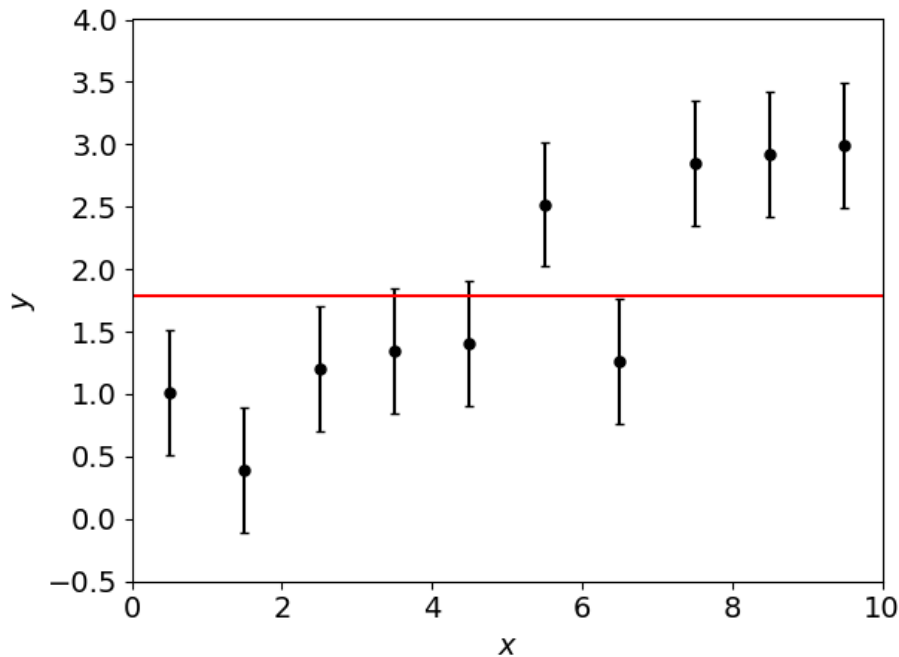


Activity: MCMC



Model selection

- Since Bayesian statistics is related to the probability of models, it allows us to perform **model selection**
- *For example: how many model parameters does a dataset justify including in a fit?*



Model selection

- A common approach is to calculate the **Akaike information criteria** for each model:

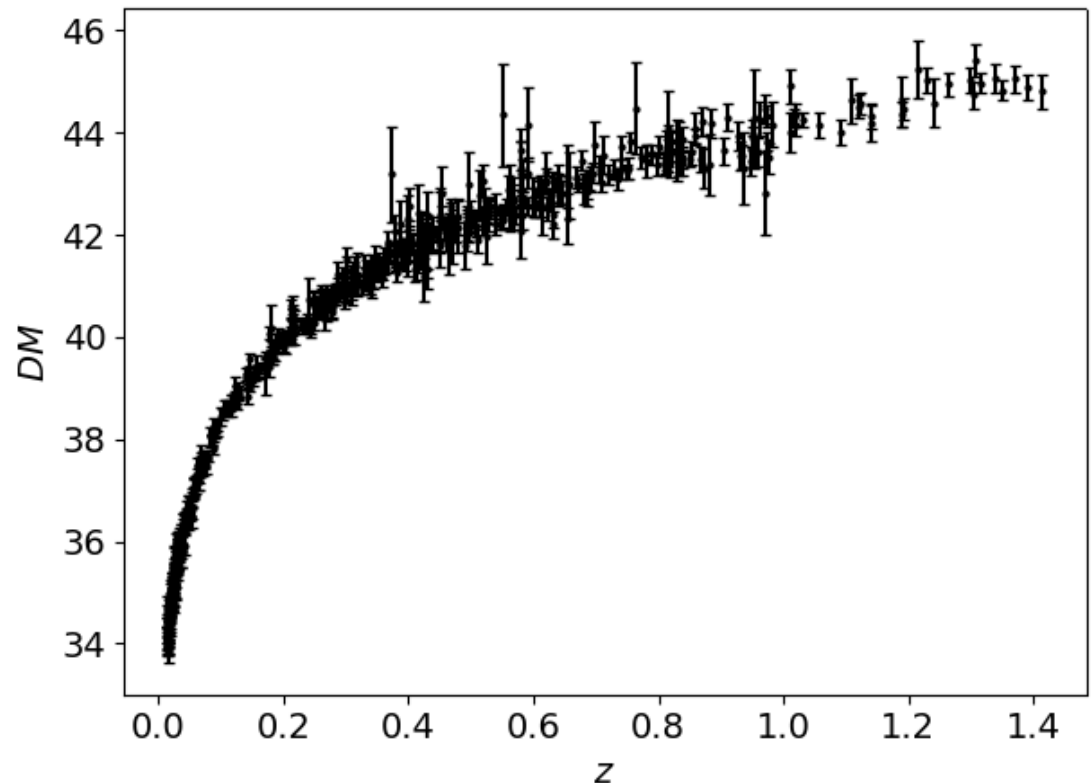
$$AIC = \chi^2_{\min} + 2p + \frac{2p(p+1)}{N-p-1}$$

p = number of parameters
 N = number of bins

- This penalizes models with more parameters (and the final term corrects for sample size)
- The model with the **smaller value of AIC is preferred** [the likelihood ratio is $e^{(AIC_1 - AIC_2)/2}$]

Activity: model selection

- Using the **supernova distance-redshift dataset**:
- Compute the **Akaike information criteria** for a **flat model** (where $\Omega_m + \Omega_\Lambda = 1$) and a **curved model** (where Ω_m, Ω_Λ can take any value). *Which model is preferred, by this metric?*



Activity: model selection

Flat Universe:

Minimum $\chi^2 = 562.2$

$AIC = 564.2$

Curved Universe:

Minimum $\chi^2 = 562.2$

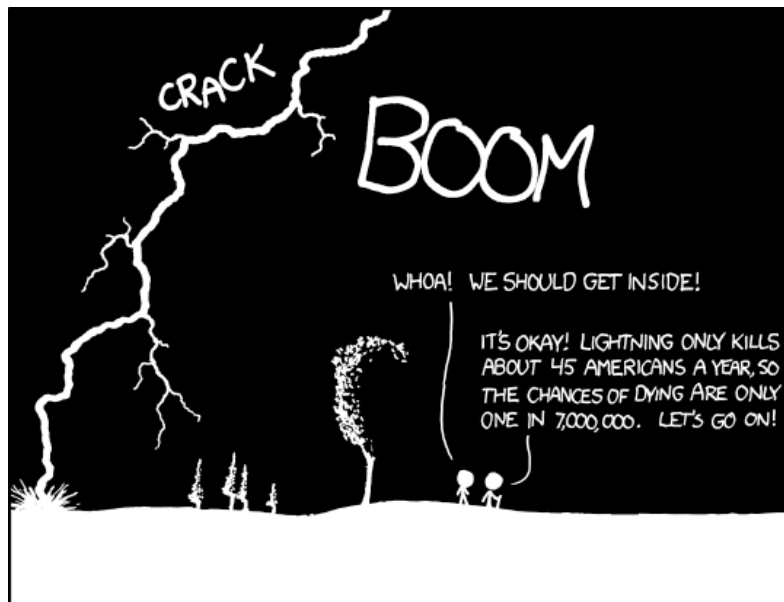
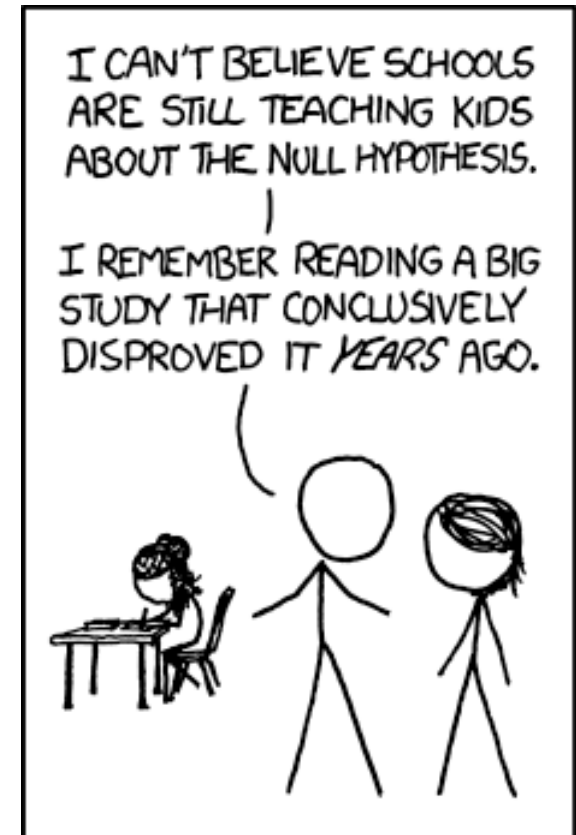
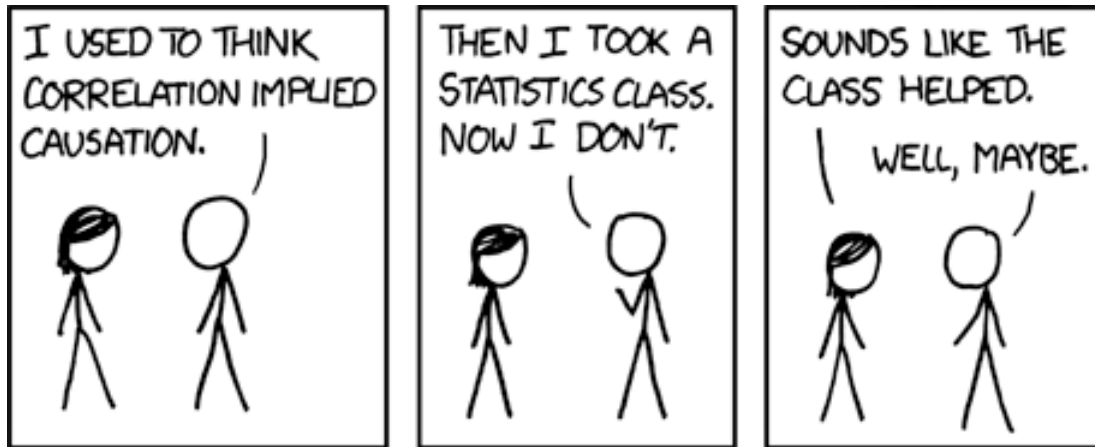
$AIC = 566.2$

Likelihood ratio = 2.7

→ **No preference for a curved Universe**

K	Strength of evidence
1 – 3	“barely worth mentioning”
3 – 10	“substantial”
10 – 30	“strong”
> 30	“very strong”

Thanks for joining stats class!



THE ANNUAL DEATH RATE AMONG PEOPLE WHO KNOW THAT STATISTIC IS ONE IN SIX.

Credit: xkcd.com